Security assessment — Claude apps gateway (AWS GovCloud), 2026-07

Point-in-time security assessment of the deployment template in this repository, prepared for the RMF/FedRAMP ATO submission. Companion to `architecture.md` (what the system is built from), `network-access-controls.md` (who can talk to what), and `conops.md` (how it is operated). Remediation tracking for everything below lives in `poam.md`; gaps in the *documentation package itself* (as opposed to the system) are tracked separately in `ato-package-gaps.md` and are deliberately not repeated here.

This deployment is a **client-configurable template**, not a single fielded system. Every organization-specific value is a CloudFormation parameter or a `scripts/deploy.env` variable; placeholders such as `<prefix>`, `<region>`, and `claude-gateway.example.com` stand in for them throughout. Findings that depend on a per-deployment value say so.

When this document and the code disagree, the code (`cloudformation/`, `docker/`, `scripts/`, `client/`) is authoritative and this document needs a PR.

---

1. Scope, methodology, and result

1.1 System state at assessment

The pilot deployment is **live and stable** as of 2026-07-28. This assessment covers the committed state of the repository at that date. It is a **documentation-only pass**: no code, template, or script change was made in response to any finding below. Every finding is therefore recorded as **Open**.

1.2 Scope

In scope — the whole repository posture:

Area	Artifacts
Infrastructure	`cloudformation/01-database.yaml`, `02-gateway.yaml`, `03-observability.yaml`, `04-download-portal.yaml`
Deploy/operate chain	`scripts/*.sh`, `scripts/common.sh`, `scripts/mirror/`, `scripts/diagnostics/`
Container images	`docker/` (gateway + entrypoint), `docker/db-admin/`, `docker/grafana/`, `docker/portal/`
Application code	the portal Flask package `docker/portal/portal/`, the db-admin bootstrap/rotation Lambda `docker/db-admin/app.py`
Client rollout	`client/Install-ClaudeCode.ps1`, `client/install-claude-code.sh`, the portal's generated `install.cmd` / `install.sh`
Telemetry / audit chain	ADOT sidecar config in 02, AMP, Grafana, the CloudWatch→Firehose→S3 activity archive, portal audit, pgaudit, Bedrock prompt logging
CI gates	`tests/cfn` (cfn-lint + cfn-guard), `tests/lambda`, `tests/portal`, `tests/bash`, `tests/powershell`

Out of scope: the Claude apps gateway binary itself (vendor-supplied, assessed only through its observable configuration schema and behavior), Okta, Zscaler/ZPA, the AWS Landing Zone, and any control the operating organization inherits from those.

1.3 Methodology

A multi-agent review was run across five independent dimensions, each with its own finder pass over the code and templates:

1. **IAM least privilege and KMS / encryption at rest** — every role, policy, trust policy, key policy, endpoint policy, and bucket policy.
2. **Network** — security groups, VPC endpoints, ALB/TLS posture, egress paths, the loopback telemetry hop.
3. **Application code** — the portal Flask package, the gateway entrypoint, the db-admin Lambda, and the operator scripts that build requests or SQL.
4. **Supply chain** — the offline build model, the mirror layer, artifact verification, image provenance, and both client installers.
5. **Monitoring, alerting, and audit completeness** (AU/SI) — alarm coverage, audit destinations, retention, and failure visibility.

Each candidate finding was **adversarially calibrated** before inclusion: the claimed AWS / Postgres / Grafana / OIDC semantics were re-checked, the exploit or failure path was traced to a concrete consequence, and findings whose premise did not survive that pass were recorded as **refuted** (§6) rather than dropped silently, so future reviewers do not re-raise them.

Findings raised by more than one dimension have been **merged into a single ID**, with both angles noted in the description. The five MEDIUM findings from the prior (2026-07-27) resubmission assessment were re-verified against the current tree; **all five remain open** and are carried forward with their original IDs noted.

1.4 Result summary

Severity	Count	Notes
Critical	0	none identified
High	0	none identified
Medium	15	includes all 5 prior-assessment mediums, still open
Low	42	includes the prior assessment's still-open lows
Informational	12	hardening opportunities, no attributable risk

The medium and low counts are higher than the prior assessment's 5/20 mainly because this pass covered dimensions the prior one sampled (the whole supply chain, the whole portal application) and because **nothing has been remediated since** — the prior open items are additive, not replaced. Three prior LOW items were re-calibrated **upward** to MEDIUM on the evidence of this pass (portal secret defaults, ALB/ECS alarm coverage, vendored wheel verification); those changes are called out in the finding text.

No finding blocks operation of the pilot. The medium set is dominated by **monitoring gaps** (six findings) and **defense-in-depth scoping** (IAM conditions, bucket policies, arch pinning) rather than by any missing primary control.

2. Accepted risks and posture decisions carried forward

These are deliberate, SSP-scoped decisions, not findings. They are stated **up front** so an assessor does not have to discover them, and they are **not** POA&M items. They are reproduced from the prior review package (the living review log and the 2026-07-27 resubmission assessment) with their rationale intact.

A1. S3 Object Lock is deferred (original finding C9)

Decision: deferred by user decision (2026-07-15); revisit if the AO requires WORM.

What the risk is. The audit/log S3 archives — the AI activity archive, the Bedrock prompt-log bucket, and the ALB access-logs bucket — carry no Object Lock and no WORM retention. A principal with sufficient S3/IAM privilege could delete objects or shorten retention (AU-9).

Compensating controls actually in place: CMK encryption (except ALB logs, see A5), IAM-only access paths, bucket versioning where applicable, `DeletionPolicy: Retain` on the buckets and on **every** CloudWatch log group in all four templates, public access fully blocked, and TLS-only bucket policies.

Note that finding SA-2026-07-04 below materially widens this accepted risk for one bucket: the activity archive has neither versioning nor a bucket policy.

A2. The former plaintext gateway→collector OTLP hop (C2) no longer exists — resolved by hop elimination, not TLS

This reverses a decision recorded in earlier review packages, so it must be surfaced, not silently dropped.

The original review accepted the gateway→ADOT-collector OTLP hop as "plaintext-but-SG-scoped." That acceptance was **withdrawn as unimplementable**: the Claude apps gateway *refuses* a non-HTTPS `telemetry.forward_to` URL unless the host is loopback, so the SG-scoped plaintext hop could never have booted.

Resolution: the ADOT collector now runs as a **localhost sidecar inside the gateway task** (`cloudformation/02-gateway.yaml`), with its OTLP receivers bound to `127.0.0.1:4317` / `:4318` and no port mappings. Telemetry never crosses a network; SC-8 is satisfied by **absence of transmission** — a stronger posture than TLS, with no new PKI, cert, CA, or key custody (no SC-17/SC-12 surface).

Alternatives considered and rejected, recorded so they are not re-proposed:

- enterprise-CA-signed collector leaf — org declined the CA dependency;
- ACM public cert + internal load balancer — adds an LB and a public-domain dependency;
- a self-managed application CA — technically sound (OpenSSL/BoringSSL enforce name constraints; the collector accepts inline `cert_pem` / `key_pem`) but rejected on ATO grounds: SC-17 shadow-PKI finding risk plus SC-12 key-custody burden.

Required side effect, itself an accepted item: `CLAUDE_GATEWAY_ALLOW_LOOPBACK=1` is set on the gateway container. The gateway's SSRF guard blocks loopback by default, so the sidecar cannot work without it. The override is **gated on telemetry being enabled**, re-permits **only** `loopback` and `unspecified`, and leaves EC2 IMDS (`169.254.169.254`), `100.100.100.200`, `fd00:ec2::254` and all link-local addresses blocked — probe-verified — and is pinned by a CI

template test that goes red if the override is removed while the loopback forward URL remains. Its one benign side effect is suppressing the gateway's startup "pod can reach cloud metadata endpoint" probe *warning* (a diagnostic, not a control).

Second side effect: the gateway **task role** is now the telemetry writer — `aps:RemoteWrite` on exactly this workspace, `logs:` actions on exactly the activity log group, and `kms:GenerateDataKey` on the CMK scoped by `kms:ViaService=aps.<region>.amazonaws.com`.

A3. Spend enforcement fails closed — an explicit availability-for-control trade

`enforcement.fail_closed_on_error: true` in the gateway's `admin:` block means a spend-store (RDS) error blocks inference with 429 rather than allowing an uncapped request. **A spend-store outage therefore halts all inference fleet-wide.** Operator decision. Recovery runbook: `cost-controls.md` §5.

Supporting facts an assessor needs:

- The `admin:` block is the master switch — the gateway runs spend enforcement only when admin is configured, and the config schema explicitly refuses `enforcement.fail_closed_on_error` without it.
- Caps are **data, not config**: rows in the `spend_limits` table set via `POST /v1/organizations/spend_limits`. **No cap rows = no enforcement**, so the stack is safe to deploy before any limits exist.
- The failure mode is self-announcing (fleet-wide 429s carrying the operator's configured `blocked_message`), which is why the associated monitoring gap (SA-2026-07-14) is calibrated MEDIUM, not HIGH.
- Interaction to note: an org-wide cap of $0 would halt the fleet the same way (see SA-2026-07-37 — no step-up confirmation on org-scope writes).

A4. Bedrock model-invocation prompt logging (opt-in) is account- and region-wide

When the org enables it (`BEDROCK_PROMPT_LOGGING=true`, tri-state: empty never touches the account setting), it captures **verbatim prompts and responses of every `bedrock-runtime` call in the account and region** — not just this gateway's. Fine in a dedicated landing-zone workload account; **wrong in a shared one.**

Stated honestly, and to be repeated in any SSP text:

1. Account+region blast radius as above.
2. **No per-user attribution** — `identity.arn` is the gateway task role for all gateway traffic. The AI activity stream remains the who-did-what audit; this is the what-was-said record.
3. **Bodies >100 KB (typical Claude Code contexts) appear ONLY in S3**, never in CloudWatch. The S3 bucket is required, not an archive nicety.

Design decisions that must not be "tidied" later:

- The destinations (CMK CloudWatch group `/claude/<prefix>/bedrock-prompts`, 14-day window; CMK S3 bucket, 731 days, `Retain`, TLS-only, `BucketOwnerEnforced`) are created **unconditionally**. Gating them on the flag breaks twice: a conditional fixed-name `Retain` log group collides on re-

enable ("log group already exists" fails the whole 03 update), and flipping the flag off tears down the delivery role and bucket grant **while the account config still points at them**, silently stopping delivery (Bedrock delivery failures do not fail invocations — an audit-continuity gap).

- The account-level `PutModelInvocationLoggingConfiguration` is applied by `deploy-observability.sh`, not a CloudFormation custom resource, because an account-level singleton tied to a stack lifecycle risks clobbering shared account state.
- The prompt bucket deliberately has **no S3 Bucket Key**: a bucket key requires the delivering service principal to also hold `kms:Decrypt`, which the docs-prescribed GenerateDataKey-only grant does not give — it would silently break delivery of exactly the >100 KB bodies only S3 holds. Test-pinned.
- The S3 grant is **bucket-wide** `s3:PutObject` rather than the documented `AWSLogs/...` prefix, because >100 KB bodies land under a separate delivery-managed "data" prefix whose path AWS does not document. The restriction is carried by `aws:SourceAccount` + `aws:SourceArn` conditions (the AWS-prescribed confused-deputy pattern, applied consistently in all three places it must be: CMK grant, bucket policy, delivery-role trust). **This remains an open improvement, tracked as SA-2026-07-58** — tighten the prefix once the live delivery path is known.
- `videoDataDeliveryEnabled` is deliberately omitted from the put JSON (older CLI service models reject the member client-side; `false` is its default).
- The disable path is get-then-delete so a standing `false` does not issue account-wide deletes (or require `bedrock:Delete*`) on every unrelated 03 re-run.
- 01's CMK policy carries the docs-prescribed `kms:GenerateDataKey` for `bedrock.amazonaws.com` (SourceAccount/SourceArn-scoped, inert until enabled). **Bring-your-own-key deployments must add this statement themselves.**

A5. The ALB access-logs bucket is SSE-S3, not CMK

The single deliberate exception to the everything-under-the-customer-CMK rule: **ELB log delivery does not support KMS.** Compensations: public access fully blocked, IAM-only reads, lifecycle expiry (90 days). Do not "fix" this.

Related: the bucket policy retains **both** ELB delivery principals — the `logdelivery.elasticloadbalancing.amazonaws.com` service principal *and* the legacy per-region ELB account — belt-and-suspenders that costs nothing and covers either writer. `BucketOwnerEnforced` accepts the legacy writer's `bucket-owner-full-control` canned ACL, so ACLs stay disabled.

A6. `ClientIngressCidr` must be tightened per deployment

Original finding B2. Addressed as documented guidance plus `deploy.env` prompts; the template default is still `10.0.0.0/8`.

Restated because the blast radius has grown: the same ALB SG now fronts the gateway, `/grafana`, **and** the portal (including its `/portal/admin` spend-cap page). All three paths independently require Okta OIDC with group checks and the ALB is internal in a no-NAT spoke — but the CIDR is the **pre-auth reachability bound** and should be set to the ZPA App Connector source range, never a /8.

The *acceptance* covers the default value. What this assessment adds as a new finding (SA-2026-07-05) is that **nothing in the template or CI rejects** `0.0.0.0/0` for this parameter.

A7. The `ecs` interface endpoint carries no endpoint policy

GovCloud does not support endpoint policies on that service; a `PolicyDocument` there fails the stack. Compensated by IAM-side scoping. Pre-check the same way before adding a policy to any new endpoint:

```
aws ec2 describe-vpc-endpoint-services \
  --service-names com.amazonaws.<region>.<svc> \
  --query 'ServiceDetails[].VpcEndpointPolicySupported'
```

A8. Client-side `Bash` (curl/wget) remains a web path for end users

The managed catch-all policy denies `WebFetch`, `WebSearch`, and `mcp__*` (bare tool names, so the tools are removed from the model's context entirely; deny rules union across scopes and cannot be re-allowed, so they are not user-overridable). Deliberately **not** denied:

- `Agent` **(subagents)** — user decision 2026-07-24; a bare `Agent` deny would block all subagents, built-in and custom, which was judged too broad.
- `Bash` — a `Bash(curl *)`-style deny was considered and not applied. The residual web path is bounded by client-side Zscaler policy.

Rationale for the denies that were applied: WebFetch fetches *locally on the client* and could only fail slowly in a no-NAT/Zscaler posture; WebSearch runs server-side and Bedrock does not expose it at all, so that deny is defense-in-depth tidiness; the MCP deny closes the remaining tool-borne web path.

A9. Session/token TTL is the deprovisioning-latency bound

A user removed from an Okta group retains:

- portal access and download capability for up to `SessionTtlHours` (default 8 h, max 24 h) — group checks are evaluated once at OIDC callback and cached in the signed session;
- spend-admin capability bounded by the gateway token (default 1 h) — the gateway re-checks the *token's* groups claim per call, not live Okta.

Standard claims-in-token semantics; the ATO-relevant action is **stating the configured TTLs in the SSP**. Compensating control: offboarded users also lose ZPA network access. One caveat that was never resolved in-repo: whether the gateway's refresh grant re-validates groups is internal to the gateway binary. (This subsumes the prior assessment's AUTH-1.)

A10. The portal holds the READ-ONLY spend key (posture change, stated candidly)

Superseding the earlier "the portal holds no gateway admin key" statement: since portal v2, 02 exports `${NamePrefix}-spend-read-key-arn` and 04 injects it as container secret `SPEND_READ_KEY` so `/portal/me` can show a signed-in user their own caps and period-to-date spend.

What remains true, and is the substance of the accepted posture:

- The portal holds **no write key** and **no gateway signing secret**.
- The read key can only list caps and period-to-date spend — compromise impact is **disclosure of per-user spend/cap data, no mutation**.
- The key is used **server-side only** and never reaches a browser.
- `/portal/me` queries only the session's own Okta `sub`.
- The **all-users** admin listing (`/portal/admin/users`) deliberately does **not** use the read key: it requires each admin's own device-flow bearer, preserving per-admin attribution and the gateway-side group re-check.
- An unset/empty `SPEND_READ_KEY` feature-gates the page off with an explanatory message.
- Deployment consequence: a 04 deploy **requires a 02 that carries the export** (upgrade order: re-run 02 before 04).

A11. Spend admins act as themselves, not as a shared key

`/portal/admin` obtains each admin's **own** gateway session token through the gateway's OAuth **device flow** (RFC 8628), so the portal stores no admin key and no signing secret; the gateway re-checks the token's groups claim on every call, and `admin_audit` records `oidc:<sub>` (the individual) rather than a shared key id. Two aligned Okta-group settings gate it: `SPEND_ADMIN_GROUPS` (02 → the gateway `admin:` block's `admin_groups`) and `PORTAL_ADMIN_GROUP` (04 → page visibility). `x-api-key` always takes precedence over bearer; empty `admin_groups` (the default) disables bearer admin entirely. `set-spend-limit.sh` plus the two generated keys remain the **break-glass** path.

Finding SA-2026-07-07 identifies a gap in the *binding* of that device-flow token to the portal session; the design posture itself is sound and worth preserving.

A12. Egress 443 to 0.0.0.0/0 from tasks

Required for Okta — public SaaS, no VPC endpoint, IP ranges not pinnable. Mitigated by port scoping, endpoint policies for all AWS-bound traffic, and the landing zone's central inspection layer.

A13. Foundation-model IAM region wildcard

`bedrock:*::foundation-model/<exact-model>` — the region wildcard is required because GovCloud geo inference profiles fan out across us-gov regions. **Model IDs are exact** (all three profile IDs plus their derived foundation-model ARNs), never `anthropic.*`, in both the task-role policy and the bedrock-runtime endpoint policy.

A14. Per-source-IP attribution is deliberately not relied upon

ZPA collapses every user to a handful of App Connector IPs (`trusted_proxies: ["<VpcCidr>"]`), so identity is carried in the Okta → gateway-session chain, not the network layer. Original finding B8; not a defect, a documented property. (Finding SA-2026-07-30 notes the unverified X-Forwarded-For parsing semantics that make this property load-bearing rather than merely conservative.)

A15. Gateway access control today is "authenticated Okta user in an approved email domain"

Okta **group** claims are not used as gateway access control. The `groups` scope is requested unconditionally, but for **per-group spend caps** (`scope_type: rbac_group` resolves against the claim), not for authorization. Per-group policy is an available, unexercised extension. The portal and Grafana *do* enforce group membership.

A16. Other standing operational trade-offs

- `TelemetryFailClosed=true` **(default) is an availability trade.** The sidecar is Essential + health-checked and the gateway waits on it HEALTHY, so a persistently failed or hung collector stops the task rather than letting the gateway serve unmonitored traffic (AU-5). `TelemetryFailClosed=false` flips the trade (availability over auditability; telemetry can gap silently) and **must be recorded in the SSP if chosen**.
- **Compound-config hazard, do not allow:** with `TelemetryFailClosed=false` AND (`AlarmSnsTopicArn` unset **or** `MissingTelemetryAlarmMinutes=0`) there is no task-stop *and* no notification on telemetry loss — total silent loss, an AU-5/AU-12 hole.
- **The missing-telemetry alarm detects total cessation, not degradation.** `Average ≤ 0` resets on any non-zero minute, so a pipeline dropping 99% of samples does not fire. It is a floor, not a coverage guarantee. (This is precisely the blindness SA-2026-07-10 names.)
- `missing-activity-logs` **is off by default and deliberately not auto-enabled with** `FORWARD_ACTIVITY_LOGS` — the audit stream is intermittent (events only on real tool use), so a short window would false-fire on idle periods. Enable only on continuously-active fleets, with a window longer than the longest expected quiet gap.
- **Deregistration delay** is a parameter with an honest trade-off: the full delay is always waited (+5 min per deploy), and streams older than the delay are still cut on deploys.
- **Grafana user persistence:** the Grafana task has no volume, so any locally-created users live in ephemeral SQLite. Okta SSO sidesteps this because identity is authoritative in Okta; the bootstrap `admin` account is break-glass only, reachable only by redeploying with `GRAFANA_DISABLE_LOGIN_FORM=false`.
- **Metric cardinality trade (accepted deliberately):** `session.id` is kept as a metric label. Active series are bounded by *concurrent* sessions (stale series age out of AMP's active-series window in minutes) — tens of series per live session against AMP's 2M default. Keeping it is mandatory for correctness.
- **Grafana numbers are observability, not the authoritative spend record.** Authoritative spend is the gateway's Postgres `spend` table, which meters inference server-side.
- **The portal's audit-page email join** does one S3 GET per distinct `oidc:` actor (serial, uncached, bounded by the page's `limit=200` fetch and in practice by the admin population) — accepted at current scale.
- **Identity-map object versions:** the artifacts bucket is versioned with no lifecycle rule, so superseded sub→email map objects persist as noncurrent versions. Add a prefix-scoped noncurrent-version expiration if that ever matters.
- **Portal static assets** (`/portal/static/`, one CSS + one JS file, no user data) are served **unauthenticated by design** — the pre-auth error and denied pages need the stylesheet.
- **Zero-JavaScript constraint was lifted** by user decision (2026-07-27): first-party JS only; CSP is `default-src 'none'; style-src 'self'; script-src 'self'; img-src 'self'; form-action 'self'; frame-ancestors 'none'; object-src 'none'`. Inline script/style remains banned.

- `MANAGED_CLI_GROUPS` **was retired** (2026-07-24): its update-lockdown payload now rides the catch-all managed policy and reaches every user — strictly broader coverage with no groups-claim dependency.
- **Client distribution is the precompiled native binary only** (decision 2026-07-15); no Node/npm distribution.
- **A fully no-admin *interactive* gateway login is not possible by design.** Claude Code's Cloud-gateway login path appears only when `forceLoginMethod: "gateway"` + `forceLoginGatewayUrl` are present in a **managed** settings source (Windows `HKLM\SOFTWARE\Policies\ClaudeCode` REG_SZ `Settings`, or `%ProgramFiles%\ClaudeCode\managed-settings.json`; Linux `/etc/claude-code/managed-settings.json`; macOS plist). These keys are never honored from user settings or HKCU. This is deliberate anti-phishing design — a user-typed gateway URL would let an attacker harvest corporate SSO. The **binary install itself stays no-admin**; only the login *config* needs the managed source, delivered by GPO/MDM.
- `requiredMinimumVersion` **fails open** on an invalid value (stripped, not enforced), so a bad push cannot brick startup fleet-wide. It is honored only from managed settings, and enforcement begins at a client's next start after a settings fetch — a ratchet, not an instant gate. Because auto-updates are locked down, the floor must only be raised **after** `publish-portal-release.sh` has the matching installer published.
- `GF_PLUGINS_PREINSTALL_DISABLED=true` is a deliberate control, not tuning: Grafana ≥12's background plugin preinstaller dials grafana.com at every boot, the same startup-egress class that crash-looped 11.x behind inspected egress.

---

3. Findings — MEDIUM (15)

SA-2026-07-01 — ALB→task TLS keys are baked into images at build time (doc misstatement corrected; key-lifecycle decision open)

Component: `docker/Dockerfile`, `docker/grafana/Dockerfile`, `docker/portal/Dockerfile`, `scripts/build-and-push-*.sh`; and the documentation set.

Description. The private key and certificate for the ALB→task hop are generated on the **build host** by the build scripts (`build-and-push-image.sh:65-68`, `build-and-push-grafana.sh:46-54`, `build-and-push-portal.sh:17-28`) and `COPY`'d into the image (`docker/Dockerfile:22-45`, `docker/grafana/Dockerfile:32-40`, `docker/portal/Dockerfile:50-54`). `docker/entrypoint.sh:37-42` says so explicitly — the entrypoint generates nothing. When this assessment ran, the ATO package stated the opposite ("per-task self-signed cert generated at startup") in `architecture.md`, `conops.md`, the architecture diagrams, in-template comments in stacks 02/03/04, and the repository README. **The documentation half of this finding was remediated in the same change that produced this assessment** — every listed claim now states build-time generation, the shared-per-image key, and image rebuild as the rotation unit, and `architecture.md` §10 records the implication as an accepted-risk entry.

Failure scenario (residual, open). The ALB→task hop key lives in an ECR image layer, is identical across every task of a deployment and across re-deployments, and persists until the next image build. Any principal that can pull the image holds the key.

Mitigating context. The ALB does not validate target certificates, and this cert never faces a developer: clients pin the ALB-presented enterprise cert. The exposure is confined to anyone who already has ECR pull rights inside the account.

Recommendation. Decide the key-lifecycle posture deliberately: either accept build-time generation as the documented design (the docs now describe it; record the ECR-pull-implies-key-access bound in the SSP and close this item), or move generation to the container entrypoint for a per-task key (requires openssl in the image, which the gateway Dockerfile deliberately avoids today). The MEDIUM rating reflected the doc-accuracy component at assessment time; the residual technical component alone is LOW.

SA-2026-07-02 — Six roles hold `kms:Decrypt` / `GenerateDataKey` on the shared CMK with no `kms:ViaService` scoping

Component: `02-gateway.yaml` (ExecutionRole 874-879, DbAdminLambdaRole 1606-1612), `03-observability.yaml` (ActivityFirehoseRole 480-486, GrafanaExecutionRole 785-789), `04-download-portal.yaml` (PortalExecutionRole 434-437, PortalTaskRole 468-474). Carried forward from the prior assessment as **IAM-1**, re-calibrated LOW → MEDIUM.

Description. One customer-managed key covers RDS storage, every secret, every log group, AMP, the activity archive, the Bedrock prompt-log bucket, and the portal artifacts bucket. Six grants of `kms:Decrypt` (and in places `GenerateDataKey`) on that key carry no `kms:ViaService` condition. The correct pattern is already present twice in the same templates — `AmpRemoteWriteKms` (02:941-948) and `GrafanaTaskRole` (03:821-829) both condition on `kms:ViaService=aps.<region>.amazonaws.com`.

Failure scenario. An unconditioned `kms:Decrypt` is a general decryption oracle for every data class the key protects. Code execution in the portal container (whose **task** role is one of the six, and is therefore container-reachable) plus any separate read path to ciphertext — an S3 object, a log export — yields plaintext of data that role has no business decrypting.

Recommendation. Add `kms:ViaService` conditions: `secretsmanager.<region>.amazonaws.com` for the execution and Lambda roles; `s3.<region>.amazonaws.com` plus `kms:EncryptionContext:aws:s3:arn` for the portal task role and the Firehose role. No functional change to any working path.

SA-2026-07-03 — Build scripts can silently create permanently non-CMK ECR repositories

Component: `scripts/common.sh:301-311` (`ensure_ecr_repo`); callers `build-and-push-image.sh:47`, `build-and-push-dbadmin.sh:27`, `build-and-push-grafana.sh:68`, `build-and-push-portal.sh:54`, `scripts/mirror/mirror-collector.sh:19`. Raised independently by the IAM/KMS and supply-chain dimensions.

Description. `ensure_ecr_repo` omits `--encryption-configuration` when `KMS_KEY_ARN` is empty and emits no warning. Only `mirror-base-images.sh:71-79` fails closed, with a named `ALLOW_NONCMK_BASE_REPOS=1` override. ECR encryption is **fixed at repository creation** — it cannot be changed later.

Failure scenario. The offline two-host model means the build host runs its own copy of `deploy.env`. If that copy lacks the `KMS_KEY_ARN` line (a routine copy-over omission, called out in the runbooks as a manual step), every repository is created AES256 and stays that way. This violates the repository's own "everything at rest uses the customer-managed key" rule, and it is invisible to the cfn-guard CI gate because these repositories are script-created, not template-created. Remediation after the fact is delete + recreate + re-push all images.

Recommendation. Move the fail-closed check into `ensure_ecr_repo` itself so all six callers inherit it, reusing the same named override; additionally, log loudly when an **existing** repository's `encryptionType` is not `KMS`.

SA-2026-07-04 — The activity-archive S3 bucket has no bucket policy and no versioning

Component: `03-observability.yaml:428-452` (`ActivityArchiveBucket`). Raised by three dimensions (network M2, IAM/KMS 6, monitoring NEW-3).

Description. No `AWS::S3::BucketPolicy` exists for this bucket at all — no `DenyInsecureTransport`, no SSE-enforcement deny — and no `VersioningConfiguration`. Its siblings are better protected: the Bedrock prompt-log bucket carries `DenyInsecureTransport` (03:623-649) and the portal artifacts bucket carries both a policy (04:280-295) and versioning (04:276). The ALB access-log bucket policy (02:712-741) also lacks the transport deny — see SA-2026-07-63.

Failure scenario. This bucket holds the 731-day durable copy of the AI activity stream, the most sensitive data class in the system (bash commands, tool inputs, file paths, per user). Absent a transport deny, any principal with read access can retrieve it over plain HTTP. Absent versioning, an accidental or malicious `DeleteObject` is unrecoverable — which materially widens accepted risk A1 (Object Lock deferred), since versioning alone would give delete-marker recovery at zero policy cost.

Recommendation. Add an `ActivityArchiveBucketPolicy` with `DenyInsecureTransport` (and consider a `Deny` on PutObject where the SSE header is not `aws:kms` with this CMK); enable `VersioningConfiguration: Enabled`; consider denying `s3:DeleteObjectVersion` to all but a break-glass principal. Add a cfn-guard rule asserting every bucket in the templates has a `SecureTransport` deny, with the named ALB-logs exception.

SA-2026-07-05 — Nothing rejects `0.0.0.0/0` for `ClientIngressCidr`

Component: `02-gateway.yaml:42-49` (parameter) → `AlbSecurityGroup` ingress on 443 (02:627-632); `tests/cfn/rules.guard`.

Description. The parameter's `AllowedPattern` is `^\d+\.\d+\.\d+\.\d+/\d+$`, which accepts `0.0.0.0/0`. The default is `10.0.0.0/8`. `deploy.env.example:42` says "NARROW THIS" but ships the /8, and no CI rule constrains ingress breadth on any security group.

Failure scenario. The pre-authentication reachability bound for the gateway `/healthz`, the gateway admin API (key-protected but not network-protected), `/portal`, and `/grafana` is whatever the operator types. A `0.0.0.0/0` is accepted without comment and means everything the landing zone routes to the ALB subnets. The /8 default means the whole enterprise RFC1918 space over the Transit Gateway.

Relationship to accepted risk A6. A6 accepts the *default value* and handles it with documented guidance. This finding is the separate observation that the template and CI provide **no gate** — the prior assessment's refutation of "the default is too broad" does not cover it.

Recommendation. Tighten the `AllowedPattern` (or add a `Rules` assertion) to reject `0.0.0.0/0` and a prefix length below some floor; add a cfn-guard rule forbidding `0.0.0.0/0` on any ingress; consider shipping an obviously-placeholder narrow default that forces a conscious choice. (Subsumes the prior assessment's **NET-1**.)

SA-2026-07-06 — Portal cookie-signing secret and OIDC client secret default to `""` instead of failing closed

Component: `docker/portal/portal/config.py:23-24`; `docker/portal/portal/crypto.py:33-61`. Carried forward from the prior assessment as **APP-2**, re-calibrated LOW → MEDIUM.

Description. `self.session_secret = env.get("SESSION_SECRET", "")` and `self.client_secret = env.get("OIDC_CLIENT_SECRET", "")` — both use `.get` with an empty default, where every other required setting on the adjacent lines uses `env[...]` and fails at boot. `crypto.py` will happily sign and verify with `""` as the key. No test asserts that the secret is required.

Failure scenario. If `SESSION_SECRET` is ever empty — a missed secret injection, a stack parameter mishap, a local-run misconfiguration promoted — every signed cookie becomes forgeable by anyone. An attacker mints a `portal_session` naming any victim `sub` and any group, obtaining: installer downloads under a forged audit identity, the admin navigation, and via `/portal/me` (which passes `user_ids=[sub]` to the gateway with the portal's read key) any user's caps and spend. Cap *mutations* still require a real gateway device-flow token, so this is an authentication bypass on the portal, not on the gateway. The stack generates a 48-character secret (04:241-249), so the failure requires a deployment error rather than a design flaw — which is why it is MEDIUM, not HIGH.

Recommendation. Use `env["SESSION_SECRET"]` with a minimum-length assertion (≥32 characters) and `env["OIDC_CLIENT_SECRET"]`; add a boot-failure test to `tests/portal` that goes red when either is absent or short.

SA-2026-07-07 — The device-flow gateway token is never bound to the portal session identity

Component: `docker/portal/portal/views/admin.py:170-194` and the two refresh paths (99-106, 446-456); `docker/portal/portal/auth.py:93`; `docker/portal/portal/views/me.py:84`.

Description. After the RFC 8628 device flow completes, the portal stores the resulting gateway token and calls `record_principal_email(gateway_token_sub(result), <session email>)` with **no equality check** between the token's `sub` and the session's `sub`. The session already carries the Okta `sub` (`auth.py:93`) and `/portal/me` already treats that value as the gateway user id (`me.py:84`), so the two identifiers are in the same namespace and the check costs nothing.

Failure scenario. The device flow is inherently phishable: admin A starts the flow in the portal and passes the `verification_uri_complete` to admin B, who approves it. A's browser cookie now holds B's gateway token. Every subsequent spend-cap change runs as B — the gateway's `admin_audit` records `oidc:<B>` — and the portal's identity map is poisoned, because `identity/principal-emails/<B-sub>.json` is overwritten with A's email. The portal's audit Email column then attributes B's actions to A, persistently, across sessions. This defeats the per-admin attribution that accepted-risk A11 is built to provide.

Note on test coverage. `tests/portal/test_admin.py:599-617` asserts the current, weaker behavior — it pairs session sub `00u123` with token sub `00uAdmin` and asserts the mapping is written. Adding a binding check will break that test; the test is wrong and should be updated with the fix.

Recommendation. Compare `gateway_token_sub(result) == session["sub"]` in `_poll_device_flow` and in both refresh paths; on mismatch, refuse the token, clear the gateway cookie, and write an audit denial. Confirm the two `sub` formats match against the live deployment before enforcing.

SA-2026-07-08 — Vendored Python wheels are the one unverified artifact class in the supply chain

Component: `docker/portal/Dockerfile:21-24`, `docker/db-admin/Dockerfile:19-22`, `docker/portal/requirements.txt`, `scripts/mirror/mirror-python-deps.sh:35-37`. Carried forward from the prior assessment as **SUP-6**, re-calibrated LOW → MEDIUM.

Description. Every other external input to the build has an integrity gate: GPG signature plus sha256 for the `claude` release, a pinned sha256 for the Grafana AMP datasource plugin, digest pins for the four base images. The 16 portal wheels and 5 db-admin wheels are committed binaries installed with `pip install --no-index --find-links /tmp/vendor -r requirements.txt` — no `--require-hashes`. `requirements.txt` pins only the three top-level packages; transitive versions are determined solely by which files happen to sit in `vendor/`, and `pip --no-index` resolves the highest matching version present.

Failure scenario. A tampered or additional `.whl` in a pull request is a binary blob no reviewer diffs, and it silently wins resolution. The code it contains executes inside the portal task, which holds the OIDC client secret, the spend read key, the session-signing material, and `s3:PutObject` on the identity prefix. The same applies to the db-admin image, which holds RDS master credentials.

Recommendation. Have `mirror-python-deps.sh` emit a committed `requirements.lock` carrying full transitive pins with `--hash=sha256:…` per wheel, and install with `--require-hashes --no-deps`. This makes any wheel substitution a visible, failing diff.

SA-2026-07-09 — `mirror-collector.sh` lacks the architecture pin and assertion its sibling establishes

Component: `scripts/mirror/mirror-collector.sh:24-26`. This is the prior assessment's **SUP-1**, confirmed still open; raised again by both the supply-chain and monitoring dimensions.

Description. The script performs a bare `docker pull` with no `--platform linux/amd64`, no post-pull architecture assertion, and pushes under a fixed `${ADOT_VERSION}` tag into an IMMUTABLE repository. `scripts/mirror/mirror-base-images.sh` documents and fixes exactly this hazard for the four base images (pin plus assert at 104-113; content-derived tag at 83-93).

Failure scenario. Mirroring from a non-x86_64 host (an Apple Silicon laptop is the realistic case) produces an arm64 ADOT image. The collector sidecar is `Essential` and health-checked, and the gateway container waits on it reaching HEALTHY — so an exec-format failure means the gateway serves nothing. Recovery is not a re-run: the IMMUTABLE repository rejects a same-tag re-push, so an operator must `batch-delete-image` manually and re-mirror. Calibrated HIGH → MEDIUM in the prior assessment because it fires only from a non-x86_64 mirror host, the impact is availability-only, and it surfaces loudly at deploy.

Recommendation. Back-port `--platform linux/amd64` plus the post-pull architecture assertion, and adopt the sibling's `<tag>-<12-hex digest>` content-derived tagging scheme so a corrected re-push is possible.

SA-2026-07-10 — The collector self-metrics heartbeat structurally blinds the only metrics-pipeline alarm to client-metric loss

Component: `02-gateway.yaml` ADOT sidecar config (`AOT_CONFIG_CONTENT`, 1443-1524); `03-observability.yaml:297` (`MissingTelemetryAlarm`). Subsumes the prior assessment's **AUD-2**.

Description. The sidecar's `prometheus` receiver scrapes the collector's own `otelcol_*` self-metrics on loopback `:8888` and feeds them into the **same** remote-write pipeline as client OTLP data. The `MissingTelemetryAlarm` fires only when total AMP ingestion reaches zero. The heartbeat therefore guarantees the alarm stays OK whenever the collector is alive, regardless of whether any client metric arrives.

Failure scenario — already realized. The 2026-07-24 delta-temporality incident: 100% of `claude_code_*` metrics were dropped at prometheus translation while every counter looked healthy (`send_failed` at 0, nothing logged). Ingestion stayed non-zero because of the heartbeat, so the alarm never fired. The only live giveaway was `otelcol_exporter_prometheusremotewrite_failed_translations` climbing — found by an operator, not by monitoring. The root cause is fixed (the managed policy now pushes cumulative temporality to every client), but the **detection gap is unchanged**, and the same blindness applies to any future drop-at-translation mode.

Implementation note the prior assessment recorded, still true. The obvious one-line fix is not implementable as written: the `failed_translations` counter lives in AMP, which CloudWatch alarms cannot evaluate. Closing this properly needs AMP rule groups plus an alertmanager→SNS path — new infrastructure.

Recommendation. Add an alarm specific to *client* data — an AMP rule on `absent_over_time(claude_code_cost_usage[…])`, or a rule on `increase(failed_translations) > 0`. Keep the existing heartbeat alarm as the pipeline-liveness backstop. If the AMP alerting path is not built, document the operator cadence for running `scripts/diagnostics/amp-query.py` as the compensating control and say so in the SSP.

SA-2026-07-11 — Firehose delivery errors are recorded nowhere; the durable activity-archive leg is unmonitored

Component: `03-observability.yaml:488-502` (`ActivityDeliveryStream`), `03-observability.yaml:350` (`MissingActivityLogsAlarm`). Merges the prior assessment's **AUD-3** with the monitoring dimension's NEW-2.

Description. The delivery stream has no `CloudWatchLoggingOptions`, so Firehose writes no diagnostic log stream at all. `ErrorOutputPrefix` routes *failed records*, but permission, KMS, and throttling failures occur before any write — they leave no trace anywhere. Separately, the optional `MissingActivityLogsAlarm` watches `AWS/Logs IncomingLogEvents` on the **CloudWatch log group**, i.e. the leg *before* the subscription filter; it stays OK whenever events reach CloudWatch regardless of whether anything reaches S3. It is also off by default (`ActivityLogsAlarmMinutes: 0`).

Failure scenario. The chain is CloudWatch (14 d) → SubscriptionFilter → Firehose → S3 (731 d). A broken Firehose role, a KMS denial, or a filter misconfiguration stops the copy that carries the two-year retention while every existing signal stays green. Discovery happens when someone needs data older than 14 days — potentially long after the evidence is gone. Even manual triage is impossible today, because there is no error record to read.

Recommendation. Enable `CloudWatchLoggingOptions` into a CMK log group, and add an alarm on `DeliveryToS3.Success < 1` (plus `DataFreshness`) wired to the existing `AlarmSnsTopicArn`.

SA-2026-07-12 — No alarm on ALB 5xx, target health, or ECS running-task count

Component: `02-gateway.yaml` — the stack defines exactly two alarms (`DbRotationFailureAlarm` 2022, `CertificateExpiryAlarm` 2046) and no `AWS/ApplicationELB` or `AWS/ECS` alarm, despite Container Insights being enabled (02:826-828). Subsumes the prior assessment's **AUD-5**, re-calibrated LOW → MEDIUM.

Description. The system has two deliberate fail-closed postures (spend enforcement, A3; telemetry, A16) whose shared primary symptom — the service is up but returning errors, or the task count has collapsed — is not monitored anywhere.

Failure scenario. The spend store degrades rather than disappears: every inference request returns 429, tasks stay running, the telemetry heartbeat keeps flowing, and the missing-telemetry alarm stays OK. The fleet is 100% down with no alarm and no notification; detection is user complaints. The same is true for a target group with zero healthy hosts behind a live listener.

Recommendation. Add `UnHealthyHostCount >= 1` and `HTTPCode_Target_5XX_Count` alarms on the gateway target group, and a `RunningTaskCount < DesiredCount` alarm from Container Insights, all wired to the existing `AlarmSnsTopicArn`. One control batch covers both fail-closed dependencies.

SA-2026-07-13 — CloudTrail is an unstated, unverified assumption

Component: none — that is the finding. No `AWS::CloudTrail::Trail` exists in any template, and the word does not appear in `architecture.md`, `conops.md`, or `greenfield-deployment.md`.

Description. The ATO package relies implicitly on AWS-side API auditing for AU-2, AU-6, and AU-9 coverage of management-plane actions, but never states that a trail is a prerequisite, never says whether one is inherited from the landing zone, and never asks for **S3 data events** on the two most sensitive stores.

Failure scenario. Object-level reads and deletes on `ActivityArchiveBucket` and `BedrockPromptLogsBucket` — the AI activity archive and the verbatim prompt/response record — are **not captured by a default management-events-only trail**. An insider retrieving or deleting those objects leaves no record in this deployment or, unless the org configured data events independently, anywhere else. Likewise CMK key-usage events.

Recommendation. Add to the org-prerequisite list in `greenfield-deployment.md` and to the ConOps control inventory: an organization or account trail must exist, must include **S3 data events for the activity-archive and Bedrock prompt-log buckets**, and should include KMS key-usage events. State explicitly which AU-2/AU-6 coverage is *inherited* rather than provided by this system. Verify the trail exists as part of the deployment checklist.

SA-2026-07-14 — No alarm covers the fail-closed spend-store dependency

Component: `cloudformation/01-database.yaml`. This is the prior assessment's **AUD-1**, confirmed still open.

Description. `enforcement.fail_closed_on_error: true` (`02-gateway.yaml:1157`) makes RDS the availability gate for all inference, fleet-wide. `01-database.yaml` defines **zero** CloudWatch alarms — no `DatabaseConnections`, no `FreeStorageSpace`, no `CPUUtilization`, no RDS event subscription, no

enhanced monitoring or Performance Insights. 01 does not even carry an `AlarmSnsTopicArn` parameter. The only alarms in the whole deployment are the two in 02 and the two in 03; `om-runbooks.md` §9 says so honestly.

Failure scenario. A storage-full or connection-exhaustion condition on the spend store halts all inference with no operator notification. Calibrated HIGH → MEDIUM because the failure mode is self-announcing — fleet-wide 429s carrying the operator's configured `blocked_message` — so the gap is paging latency and MTTR, not silent loss, and the operations documentation discloses it rather than claiming coverage.

Recommendation. Add RDS alarms to 01 (free storage, connections, CPU, and an RDS event subscription for failover/failure events), wired to the same `AlarmSnsTopicArn` that 02 and 03 use — which requires adding that parameter to 01. Then correct `cost-controls.md` §5, which currently points readers at "the DB-side alarms in om-runbooks.md §9" that do not exist.

SA-2026-07-15 — Portal audit writes fail open, silently, with no alarm surface

Component: `docker/portal/portal/audit.py:71-82` and `_ensure_stream`; `04-download-portal.yaml:304-313` (`AuditLogGroup`); `scripts/deploy-download-portal.sh`. This is the prior assessment's **AUD-4**, re-located to the v2 Flask package and confirmed still open.

Description. `AuditLogger.write()` swallows every `put_log_events` exception with a `log.error()` into the **operational** log group — not the SIEM-flagged `portal-audit` group, which is precisely the write that failed — and `_ensure_stream` swallows its failures at debug level. A permanent error such as `AccessDenied` therefore silences every subsequent audit write for the life of the task. Stack 04 defines no alarms and no metric filters at all, and `deploy-download-portal.sh` never passes `ALARM_SNS_TOPIC_ARN`, so the stack has no alarm surface to attach one to.

Failure scenario. Installer downloads and access denials — the portal's whole audit trail — stop being recorded, and nothing indicates it. Moderating fact: admin spend-cap actions are independently recorded in the gateway's Postgres `admin_audit` table, so only download and denial events are single-trail.

Recommendation. Give stack 04 an alarm surface (`AlarmSnsTopicArn` parameter, passed by the deploy script); emit a CloudWatch **metric** on audit-write failure and alarm on it; at minimum, escalate the `_ensure_stream` failure from debug to error and re-raise on repeated `put_log_events` failure rather than continuing indefinitely.

---

4. Findings — LOW (42)

Grouped by assessment dimension. Each row is independently actionable; none has a demonstrated exploitation path at current configuration.

4.1 IAM, KMS, and encryption at rest

ID	Component	Finding	Recommendation
SA-2026-07-16	`common.sh:327-341` (`ensure_artifacts_bucket`)	The CloudFormation deploy-artifacts bucket is created outside the templates with SSE-S3, no TLS-only policy, and no versioning; used by `deploy-observability.sh:91` among others. It holds rendered templates carrying Okta IDs, the FQDN, model IDs, SG/subnet IDs, and rendered managed-settings. Invisible to cfn-guard.	Use SSE-KMS when `KMS_KEY_ARN` is set; add `DenyInsecureTransport` and versioning; add the bucket to the encryption inventory in `architecture.md`.
SA-2026-07-17	ExecutionRoles at 02:857-858, 03:773-774, 04:418-419	All three ECS execution roles attach the AWS managed `AmazonECSTaskExecutionRolePolicy`, which grants `logs:CreateLogStream` / `PutLogEvents` on `Resource: "*"`. Each execution role can therefore append events to **any** log group, including the CMK-encrypted audit trails (AU-9 record-integrity). Mitigated to LOW because execution-role credentials are not exposed to the container.	Replace with an inline policy scoping ECR to `<prefix>-*` repositories and `logs:` to each stack's own log-group ARNs.
SA-2026-07-18	`03-observability.yaml: 454-463`	`ActivityFirehoseRole`'s trust policy trusts `firehose.amazonaws.com` unconditionally, while both siblings are scoped — `ActivitySubscriptionRole` (03:504-516) has `aws:SourceAccount`, `BedrockPromptLoggingRole` (03:655-670) has both `SourceAccount` and `SourceArn`. The role carries S3 read on the archive plus unconditioned `kms:Decrypt`. No concrete cross-account path exists (a Firehose role must be same-account), so this is consistency and defense-in-depth.	Add `aws:SourceAccount` and `aws:SourceArn` to match the siblings.
SA-2026-07-19	`03-observability.yaml: 488-502`	The activity delivery stream has no `DeliveryStreamEncryptionConfigurationInput`, so up to 300 s / 64 MB of buffered per-user activity content sits at rest under an AWS-owned key rather than the deployment CMK. Delivered S3 objects **are** CMK-encrypted via the bucket default, so only the buffer leg is affected. Also raised by the monitoring dimension (NEW-6).	Either set `CUSTOMER_MANAGED_CMK` against the stack CMK — noting it adds an availability dependency (Firehose discards records after 24 h of `KMSAccessDenied`), requires `kms:CreateGrant` for the deployer, and needs a live check of whether the CloudWatch Logs subscription principal needs added KMS permissions for a CMK DirectPut stream — or record this leg explicitly as AWS-owned-key in the encryption inventory.
SA-2026-07-20	`docker/db-admin/app.py: 285-296`; `02-gateway.yaml: 1724-1730`	The rotation handler takes `event["SecretId"]` with no comparison to `APP_SECRET_ARN` and no `RotationEnabled` check; `DbRotationInvokePermission` is scoped by `aws:SourceAccount` only, with no `SourceArn`. Any in-account principal able to attach rotation to another secret can drive the function; the write path is contained by IAM, but the function will describe/get the master secret first. Also raised by the application-code dimension.	Assert `event["SecretId"] == APP_SECRET_ARN` and check `RotationEnabled`; add `SourceArn` to the invoke permission; add `tests/lambda` cases for both.
SA-2026-07-21	`02-gateway.yaml:949-957`	The telemetry `ActivityLogWrite` grant is gated on `HaveTelemetry` only, while the activity stream itself is independently controlled by	Introduce `WantActivityLogs = HaveTelemetry AND`

ID	Component	Finding	Recommendation
		`ForwardActivityLogs` (02:405-415). Every telemetry-enabled deployment — including the majority that leave the activity stream off — grants write access to the audit log group, permitting forged entries.	`ForwardActivityLogs` and gate the statement on it; drop the unused `DescribeLogStreams` action.
SA-2026-07-22	`02-gateway.yaml:1915-1926` (Logs endpoint), `02:1998-2014` (S3 gateway endpoint)	Two endpoint policies are wider than needed: the Logs endpoint permits `logs:PutRetentionPolicy` on `…:*` (nothing in-VPC needs it), and the S3 gateway endpoint permits `s3:*` on `*`, account-conditioned only. Additionally the `CfnCustomResourceResponses` statement (02:1998-2006) uses `Principal: "*"` with no `aws:ResourceAccount` condition against buckets matching `cloudformation-custom-resource-response-*` — a prefix that is not reserved, so a third party could own a matching bucket. No role today holds broad `s3:PutObject`. Raised by both the IAM and network dimensions.	Drop `logs:PutRetentionPolicy`; enumerate the S3 actions actually used or scope to bucket ARNs; constrain the CFN-response statement to regional CloudFormation response buckets or add an account guard. At minimum, document the IAM-side reliance in the endpoint-policy table.
SA-2026-07-23	`02-gateway.yaml` Secrets Manager endpoint policy	The policy's `secret:rds!*` pattern matches every RDS-managed secret in the account, not just this deployment's. Carried forward from the prior assessment as **IAM-2**.	Import 01's `DBMasterSecretArn` export and scope to it.
SA-2026-07-24	`common.sh` (`ensure_ecr_repo` Lambda-pull branch)	The repository policy granting `lambda.amazonaws.com` image pull scopes by `aws:SourceAccount` only, with no `aws:SourceArn`. Carried forward from the prior assessment as **IAM-3**.	Add `aws:SourceArn` for the specific function ARN.

4.2 Network

ID	Component	Finding	Recommendation
SA-2026-07-25	`02-gateway.yaml:1769-1790` (`EndpointSecurityGroup`), attached at 02:1814-1815 and ingressed from 03:395-405, 04:395-405	One shared endpoint security group fronts the bedrock-runtime endpoint **and** ecr.api / ecr.dkr / logs / secretsmanager / ecs. Its ingress admits the gateway service, the db-admin Lambda, the admin host, Grafana, and the portal — so all of those have *network* reach to the Bedrock data plane. Only IAM and the endpoint policy (`Principal: "*"`, three models) stop them; the network layer contributes nothing.	Give `BedrockRuntimeEndpoint` a dedicated SG admitting only `ServiceSecurityGroup`; update the inventory in `network-access-controls.md`.
SA-2026-07-26	02:671-678, 03:702-709, 04:350-357; also 02:1759-1764	When `HttpsProxyPort` is set, three security groups open that arbitrary TCP port to `0.0.0.0/0` egress; the db-admin Lambda egresses 443 to `0.0.0.0/0`. Accepted risk A12 covers 443-to-Okta; the arbitrary proxy port is a wider hole than A12 describes.	Add an optional `ProxyCidr` parameter to scope proxy-port egress, falling back to the current behavior with a comment when unset.
SA-2026-07-27	`02-gateway.yaml:58-61`, consumed at 02:813; `tests/cfn/rules.guard`	`TlsSecurityPolicy` is unconstrained free text with no `AllowedValues`, and no CI rule enforces a FIPS or TLS 1.2 floor on the listener. The default is correct (`ELBSecurityPolicy-TLS13-1-2-FIPS-2023-04`); nothing prevents an operator from weakening it. `desync_mitigation_mode` is also not pinned (02:766-778).	Add `AllowedValues` (the FIPS-2023-04 and TLS13-1-2-2021-06 policies); add a guard rule asserting `SslPolicy` is in the approved set and `Protocol: HTTPS` on every listener; consider pinning the strictest desync mode.
SA-2026-07-28	`02-gateway.yaml:1443-1524` (`AOT_CONFIG_CONTENT`)	The ADOT sidecar explicitly binds its OTLP receivers to `127.0.0.1:4317` / `:4318` and its health-check extension to `127.0.0.1:13133`, and self-scrapes `127.0.0.1:8888` — but never sets `service.telemetry.metrics.address`, so the self-metrics endpoint's bind address is inherited from the collector build default. Under `awsvpc` the task shares one ENI, so a future default of `0.0.0.0:8888` would be reachable at the task IP. The service security group currently prevents that.	Set `service.telemetry.metrics.address: 127.0.0.1:8888` explicitly, matching the deliberate loopback pinning of every other listener in the config.
SA-2026-07-29	`tests/cfn/rules.guard`	No CI gate asserts security-group source or CIDR **breadth**; the existing rules assert only that explicit egress is declared. Carried forward from the prior assessment as **NET-1**; the concrete instance is SA-2026-07-05.	Add guard rules for `0.0.0.0/0` on ingress and for an overly-short prefix on `ClientIngressCidr`.
SA-2026-07-30	`02-gateway.yaml:1065`	`trusted_proxies` is set to the entire `${VpcCidr}`, and the gateway binary's X-Forwarded-For parsing semantics (leftmost-untrusted versus rightmost-trusted) were never confirmed against the binary. The ALB appends to XFF and no `xff_header_processing` attribute is set. If the gateway takes the leftmost entry, a client-supplied XFF header spoofs the source IP recorded in audit and session records. The portal gets this right — `gunicorn.conf.py` reads only the last XFF entry. Accepted risk	Test a spoofed XFF header against the deployed gateway; narrow `trusted_proxies` to the ALB subnets; if unresolved, state in the SSP and to the SIEM that the recorded client IP is untrusted.

ID	Component	Finding	Recommendation
		A14 already says per-source-IP attribution is not relied upon, which bounds the impact.	

4.3 Application code (portal, Lambda, operator scripts)

ID	Component	Finding	Recommendation
SA-2026-07-31	`docker/portal/portal/common.py:59-66` ; `auth.py:45,57`	Portal cookies are `HttpOnly; Secure; SameSite=Lax; Path=/portal` with **no** `__Host-` **prefix**, and the anti-forgery `state` lives in a cookie. `crypto.py:166-177` documents the sibling-domain threat as accepted. A hostile sibling host under the same registrable domain can *set* `portal_txn` (HttpOnly prevents reading, not writing) and navigate the victim to the callback with the attacker's code and state — logging the victim in as the attacker and corrupting the download audit trail.	Move cookies to `Path=/` with the `__Host-` prefix; clear `portal_txn` on the callback failure branches (`auth.py:47-58,64-69`).
SA-2026-07-32	`docker/portal/portal/views/me.py:84,106-108`	`/portal/me` requests `user_ids=[sub]` with the read key and renders the response verbatim; the returned items carry `scope.user_id` / `actor.user_id` which are never compared to the session `sub` . The `user_ids[]` contract is binary-verified against gateway 2.1.211, but a gateway upgrade that renames or ignores the parameter would silently turn the page into an unfiltered dump of every user's caps and spend to every authenticated user.	Filter `doc["data"]` to items whose `scope` / `actor` user id equals the session `sub` , and raise an explicit error (not an empty page) if foreign rows are returned, so API drift surfaces loudly; add a test.
SA-2026-07-33	`docker/portal/portal/artifacts.py:280-287` ; generated `install.cmd` (:46) and `install.sh` (:80); `04-download-portal.yaml:131-138`	The release-manifest checksum is returned with no format validation and spliced unquoted into the generated batch and bash installers. A principal with artifacts-bucket write access could set the checksum to `deadbeef & calc.exe` — `&` is a batch separator, so the generated `install.cmd` executes attacker commands on every workstation that runs it (`;` / `$()` for bash). Today the only gate is `publish-portal-release.sh:46-61` , outside the portal. Same class: `RELEASE_VERSION` has no `AllowedPattern` and is concatenated into S3 keys (`views/downloads.py:125`).	Reject anything not matching `^[0-9a-fA-F]{64}$` in `release_sha256` and fail with a rendered error; add an `AllowedPattern` to `RELEASE_VERSION` .
SA-2026-07-34	`docker/db-admin/app.py:117-130,238,322`	Secret-derived identifiers are inlined into SQL via f-strings in `CREATE ROLE` / `ALTER ROLE` as the RDS master user, with no allowlist check against `APP_USERS` . The values come from the secret JSON, not from configuration. Passwords are safe (`ExcludePunctuation`) and `PutSecretValue` is scoped to one secret, so this is not remotely reachable — but any principal able to write the app secret obtains arbitrary SQL as master via a crafted `"username"` . `_ensure_roles` is also reached from bootstrap with the live secret's username, so this is not rotation-only.	Assert `username in APP_USERS` before any interpolation; add `tests/lambda` cases.
SA-2026-07-35	`scripts/set-spend-limit.sh:171-172`	The request body's scope JSON is built by string interpolation and `--id` is never validated. The portal's equivalent (`build_spend_limit_body`) rejects control characters and over-long values and uses `json.dumps` . An operator pasting an id containing `"` on this break-glass path can silently convert a per-user cap into an org-wide one — which, at $0, halts the fleet (A3).	Validate `--id` (non-empty, ≤320 characters, no control characters, quotes, or backslashes), or build the body with `jq -n --arg` .
SA-2026-07-36	portal (no `/portal/logout` route); `_security_headers`	There is no logout route and no session revocation: the signed cookie carries a groups snapshot for up to `SESSION_TTL_HOURS` (default 8), so a user removed from a group keeps portal-side privileges until expiry and cannot terminate a session on a shared machine. No `Strict-Transport-Security` header is	Add a CSRF-protected `POST /portal/logout` clearing both cookies; add HSTS to `_security_headers` ; consider a

ID	Component	Finding	Recommendation
		sent anywhere, so a first visit over `http://` is downgradeable. The gateway re-checks admin group membership per call, so cap mutations are covered; downloads, `/portal/me`, the guide, and admin navigation are not. Relates to accepted risk A9.	shorter TTL or re-validating groups on admin page loads.
SA-2026-07-37	portal `/portal/admin`	No step-up confirmation exists for **org-wide** spend-cap writes within an already-connected admin session. Carried forward from the prior assessment as **AUTH-2**; it is the human-error path into the A3 fail-closed outage (an org-wide cap of $0 halts the fleet).	Require an explicit typed confirmation for org-scope writes.
SA-2026-07-38	portal admin endpoints	Failed CSRF checks on admin endpoints are not audited, although group-membership denials are. Carried forward from the prior assessment as **AUTH-3**.	Audit CSRF failures on the same path as group denials.
SA-2026-07-39	`03-observability.yaml` Grafana group parameters	The Grafana group parameters lack the `AllowedPattern` their gateway equivalents carry, and are spliced into a JMESPath expression. Carried forward from the prior assessment as **AUTH-4**.	Add matching `AllowedPattern` constraints.
SA-2026-07-40	portal cookie handling	The oversized-cookie guard written for `portal_gw` was never applied to `portal_session`, so a user in very many Okta groups can hit a silent login loop. Carried forward from the prior assessment as **AUTH-5**.	Apply the same size guard to `portal_session`, with an explicit error rather than a loop.
SA-2026-07-41	portal token-exchange error path	The OAuth `code` and `state` are logged verbatim when the token exchange fails, because the catch-all handler logs the full request path. Carried forward from the prior assessment as **APP-1**.	Redact the query string in the error path.
SA-2026-07-42	portal responses	Authenticated pages — including the admin and audit views — are sent without `Cache-Control: no-store`. Carried forward from the prior assessment as **APP-3**.	Add `Cache-Control: no-store` to authenticated responses.
SA-2026-07-43	portal / gunicorn	No socket read timeout is configured, so slow clients can hold worker threads. Carried forward from the prior assessment as **APP-5**.	Set a read timeout.
SA-2026-07-44	`docker/portal/portal/views/gateway.py:216-231`	The admin's live gateway bearer **and** refresh token ride in a signed-but-**unencrypted** cookie. `HttpOnly`, the CSP, and a TTL of `min(token exp, session exp)` mitigate it; the residual is a bearer token at rest in the browser. Carried forward from the prior assessment as **KEY-1**.	Encrypt the cookie payload, or move to a server-side handle so no bearer is at rest client-side.

4.4 Supply chain, build, and installers

ID	Component	Finding	Recommendation
SA-2026-07-45	`scripts/publish-portal-release.sh:46-64` ; `scripts/build-and-push-image.sh:16-22` ; `scripts/mirror/mirror-claude-release.sh:48-54`	Re-verification compares an artifact against metadata that travels **with** it: `publish-portal-release.sh` verifies binaries against `${SRC}/manifest.json` on the same share, and `build-and-push-image.sh` re-verifies `claude` against a `CHECKSUMS.txt` the mirror wrote into the same directory. The mirror stages `manifest.json.sig` but nothing downstream uses it. An attacker with write access to the transfer share rewrites binary and manifest coherently and both checks pass — the controls are honest against corruption, not against the tampering their names imply.	When `ANTHROPIC_GPG_KEY` and `manifest.json.sig` are present, re-run `gpg --verify` in `publish-portal-release.sh` before the checksum comparison, failing closed with the same named `ALLOW_UNVERIFIED_MANIFEST=1` escape; prefer `manifest.json` over `CHECKSUMS.txt` in `build-and-push-image.sh` .
SA-2026-07-46	`scripts/build-and-push-image.sh:65-68` , `scripts/build-and-push-grafana.sh:53-56`	Two of the three TLS-key generators use a `umask 077` subshell **without** the `rm -f` the repository's own scripts rule mandates; `build-and-push-portal.sh:25-29` does it correctly. A pre-existing 0644 `server.key` rewritten in place stays world-readable and is baked into the image at that mode. The key is per-build and only covers the ALB hop, so impact is LOW — but it is a self-declared rule violation in two of three sites. Carried forward from the prior assessment as **KEY-2**.	Add `rm -f` for both the key and the certificate inside both subshells.
SA-2026-07-47	`client/Install-ClaudeCode.ps1:265-272`	The Windows installer skips SHA-256 verification **silently** when `-Sha256` is absent (`if ($Sha256) { … }` with no `else`). Its Linux twin warns explicitly (`install-claude-code.sh:167-169`). Hand-run installs — the fileshare rollout the script's own `.EXAMPLE` documents — therefore complete with zero integrity checking and no indication.	Emit `Write-Warning` when `-Sha256` is empty, mirroring the Linux script, or fail unless a named opt-out switch is passed.
SA-2026-07-48	`client/Install-ClaudeCode.ps1:280-285` ; `docker/portal/portal/artifacts.py:44-57`	The Authenticode check is a subject-substring match on a CN prefix of "Anthropic". A `-SignerThumbprint` parameter exists but the portal never passes it and no configuration knob exposes it. Any certificate that chains to a trusted root with a CN starting "Anthropic" — including an internal or TLS-inspection CA's code-signing certificate — passes. SHA-256 remains the real control, so this is a degradation of defense-in-depth.	Add an optional 04 parameter (`PortalSignerThumbprint` → `PORTAL_SIGNER_THUMBPRINT`) that `build_install_cmd` bakes into the generated installer.
SA-2026-07-49	`docker/portal/portal/views/downloads.py:90` , `artifacts.py:280-287` ; `client/install-claude-code.sh:21-24`	On Linux the only integrity control is self-referential: `install.sh` 's SHA-256 comes from `releases/<ver>/manifest.json` in the **same bucket** the binary streams from. Bucket-write access replaces binary and manifest coherently and every Linux install verifies clean. Windows	Either record as an accepted risk bounded by the bucket-write control, or publish a GPG-verified `manifest.sig` alongside and compare against a value not sourced from the same mutable prefix.

ID	Component	Finding	Recommendation
		retains Authenticode as a second layer; Linux has none. The installer states this honestly.	
SA-2026-07-50	`scripts/mirror/mirror-python-deps.sh:13-15,35-37`	The script's stated contract is universal wheels only, but `pip download --only-binary=:all: --python-version 3.12` is run with no `--platform` / `--abi` / `--implementation`. The contract is already violated in-tree: `docker/portal/vendor/markupsafe-3.0.3-cp312-…manylinux…x86_64.whl` is platform-specific. A refresh run on macOS or arm64 vendors the wrong wheel and the offline linux/amd64 build fails late on the hardened host — fail-closed, but noisily and far from the cause.	Add `--platform manylinux2014_x86_64 --implementation cp --abi cp312`; restate the contract in the script header.
SA-2026-07-51	`build-and-push-image.sh:71`, `-grafana.sh:73`, `-dbadmin.sh:32`, `-portal.sh:59`	None of the four image builds passes `--platform linux/amd64` or asserts the built image's architecture before push, while every task definition pins `X86_64`. On an arm64 build host with floating upstream base-image defaults this produces an arm64 image under an immutable tag: the task will not start and recovery requires manual ECR deletion. `mirror_one` in `mirror-base-images.sh` is the model to copy.	Add `--platform linux/amd64` to all four builds plus a pre-push `docker inspect` architecture assertion.
SA-2026-07-52	`deploy.env.example:78-81`; the four build scripts	The mirrored base-image override is unenforced: the `*_BASE_IMAGE` variables default to empty and the build scripts fall back to floating public tags. On the hardened host this fails loudly; on any host that *does* have egress it silently builds against an unpinned upstream image. Carried forward from the prior assessment as **SUP-2**; the supply-chain pass raised the same point as SC-11 (nothing warns when a base image resolves to a floating tag).	Emit a warning — or fail behind a named override — when the resolved base-image reference lacks an `@sha256:` digest.
SA-2026-07-53	`.github/workflows/tests.yml`	The cfn-guard installer is piped from a floating `main` branch into `sh`, unverified, and `cfn-lint` is unpinned. Carried forward from the prior assessment as **SUP-3**.	Pin the installer to a released tag and verify its checksum; pin `cfn-lint`.
SA-2026-07-54	CI test dependencies	Test dependencies are declared with open-ended `>=` constraints, with no lockfile and no hashes. Carried forward from the prior assessment as **SUP-4**.	Add a lockfile with hashes for the CI toolchain.

4.5 Monitoring and audit

ID	Component	Finding	Recommendation
SA-2026-07-55	`02-gateway.yaml:2046-2065` (`CertificateExpiryAlarm`), `02:2022` (`DbRotationFailureAlarm`)	The certificate-expiry alarm sets `TreatMissingData: ignore` with no `InsufficientDataActions`, so it holds OK indefinitely if ACM stops publishing `DaysToExpiry` — the alarm can stop monitoring silently. Both 02 alarms set `AlarmActions` only, with no `OKActions`, so neither signals recovery; the two alarms in 03 (328, 377) set both.	Add `InsufficientDataActions` to the certificate alarm; add `OKActions` to both 02 alarms for parity with 03.
SA-2026-07-56	`01:86` (`PgauditLogRetentionDays`), `03:218` (`ActivityLogWindowDays`), `03:232` (`BedrockPromptLogWindowDays`), `04:162` (`PortalAuditRetentionDays`), `04:170` (`PortalLogRetentionDays`)	All five retention parameters are plain `Number` with `MinValue: 1` and no `AllowedValues`, but CloudWatch Logs accepts only an enumerated set of retention values. An operator entering, say, 100 days gets a mid-deploy failure on an audit-bearing stack.	Add `AllowedValues` matching the CloudWatch Logs retention set to all five.
SA-2026-07-57	`03-observability.yaml:561-680`	Bedrock prompt-log delivery (opt-in, accepted risk A4) has no delivery monitoring: no alarm on `IncomingLogEvents` for the prompt log group and nothing on the S3 leg. Verification today is a manual `aws logs tail`. A bring-your-own-key deployment that omits the CMK grant sees delivery stop silently, and Bedrock delivery failures do not fail invocations.	Add an `IncomingLogEvents` alarm on the prompt log group (enabled only when the feature is on) and document the manual S3-leg check in `om-runbooks.md` §11.

---

5. Findings — INFORMATIONAL (12)

No attributable risk at the current configuration; recorded so a future change does not silently turn one into a finding.

ID	Component	Observation
SA-2026-07-58	`03-observability.yaml` `BedrockPromptLogsBucketPolicy` (616-640)	The Bedrock prompt-logs bucket grant is bucket-wide `s3:PutObject` (`${Bucket.Arn}/*`) where the AWS-documented template is prefix-scoped, because the >100 KB delivery "data" prefix is undocumented. The confused-deputy conditions (`aws:SourceAccount` + `aws:SourceArn`) are present and correct, and the delivery role grants exactly the fixed stream (03:671-680), so there is no gap today. **This is the one carried-forward open TODO from the code**, with the comment retained in the template; closure is to capture the real prefix from the first live delivery and scope to it. See accepted risk A4.
SA-2026-07-59	`01-database.yaml:111-153`	A single CMK covers every data class, with a root-delegated key policy (`AccountAdminViaIam`, `kms:*` to root) plus the CloudWatch Logs and Bedrock service grants. Key separation is entirely IAM-side. Either state the shared-key blast radius as an accepted risk in the SSP, or split a second CMK for the prompt-log and activity-archive stores with an explicit principal allowlist.
SA-2026-07-60	`tests/cfn/rules.guard`	There is no S3-encryption rule, so a future bucket added without SSE-KMS passes CI. Add one, with a named exception for `AlbLogsBucket` (accepted risk A5).
SA-2026-07-61	`03-observability.yaml:981-983`	The Grafana break-glass admin password is injected unconditionally, even when `GrafanaDisableLoginForm=true`. Consider gating the injection on the condition so the credential is absent when unusable.
SA-2026-07-62	`02:797`, `03:999`, `04:597`	Three unauthenticated health endpoints (`/healthz`, `/grafana/api/health`, `/portal/healthz`) are reachable by anything inside `ClientIngressCidr`. This is required for ALB health checks; it warrants one line in the SSP. Likewise: **no WAF is deployed on the ALB** — the ALB is internal-only, which should be stated as a decision rather than left implicit.
SA-2026-07-63	`02-gateway.yaml:711-741` (`AlbLogsBucketPolicy`)	The ALB access-log bucket policy carries only the two ELB delivery grants and no `DenyInsecureTransport`, unlike its siblings. SSE-S3 on this bucket is the correct documented exception (A5) and is **not** a finding; the missing transport deny is an inconsistency worth closing.
SA-2026-07-64	`docker/portal/portal/views/__init__.py:14-16`	The CSP has no `base-uri` directive. `base-uri 'none'` is free and closes base-tag injection as a class.
SA-2026-07-65	portal OIDC callback	OAuth authorization codes land in ALB access logs via the callback query string. Bounded by one-time code use, S256 PKCE, and confidential-client authentication; worth one SSP line.
SA-2026-07-66	`docker/portal/portal/templates/admin_pending.html:8`	The device-flow `verify_url` is rendered as an `href` with no scheme allowlist. The value comes from trusted infrastructure (the gateway's own device-authorization response), but a `javascript:` value would execute on click.
SA-2026-07-67	portal audit and download paths	Audit-write amplification and absent rate limiting: every non-admin GET to `/portal/admin` writes a denial record, and `/portal/download` streams 100+ MB with no throttle, so an authenticated user can inflate logging cost or saturate the task.
SA-2026-07-68	portal generated installers	Operator-configured team and cost-center values are not screened for `"` before being spliced into the generated `install.cmd`. Shell metacharacters *are* rejected at boot and per request (`selection.py:33-97`); the double-quote case is the residual. Carried forward from the prior assessment as **APP-4**.

ID	Component	Observation
SA-2026-07-69	`.github/workflows/tests.yml`	Pester is installed unpinned; `-SkipPublisherCheck` is inert on Linux runners. Carried forward from the prior assessment as **SUP-5**.

---

6. Claims raised and refuted

Recorded so future reviewers do not re-raise them. Each was proposed during a finder pass (or a prior assessment) and did not survive adversarial verification.

1. **"The S3 gateway-endpoint policy (`s3:*`, account-bounded) is overly broad."** Refuted: `aws:ResourceAccount` is the AWS-prescribed account-scoping pattern for S3 gateway endpoints, and the sibling endpoint policies are account-granular on the resource axis too; this is the documented C5 resolution. The residual — enumerating the action axis — is informational and is carried in SA-2026-07-22; the originally proposed fix (scoping to specific bucket ARNs) is impossible as written, because the 03 and 04 buckets do not exist when 02 deploys and the service-written buckets never transit the endpoint.
2. **"The `ClientIngressCidr` default is too broad."** Real, but already tracked as accepted risk A6 with its grown blast radius — not a new finding. What *is* new, and is filed as SA-2026-07-05, is that nothing gates `0.0.0.0/0`.
3. **"The spend-admin keys have no documented rotation."** Refuted: `cost-controls.md` §7 documents the full manual rotation procedure (file-based write plus forced deployment), matching the repository's documented-manual-rotation posture for all non-DB secrets. The residual — the `om-runbooks.md` §7 inventory table omitted the two keys — was fixed in the 2026-07-27 change.
4. **"pgaudit and portal-audit lack an S3 durable copy, asymmetric with the activity stream."** Refuted: the S3 leg exists only where the CloudWatch window is deliberately short. pgaudit's CloudWatch group itself carries the full 731 days and portal-audit carries 365, so retention parity exists in one durable store. The deletion-resistance angle is accepted risk A1 (Object Lock deferred). Residual: state the CloudWatch-only posture in the SSP.

---

7. Verified strengths

Confirmed against the code during this pass. Listed because an assessment that reports only gaps misrepresents the posture.

Identity and access

- The gateway task role grants `bedrock:InvokeModel*` on exactly the three configured inference-profile ARNs and their derived foundation-model ARNs — never `anthropic.*` — plus `aps:RemoteWrite` on one workspace and `logs:` on one group. It holds no secrets, no ECS, and no IAM permissions. The bedrock-runtime endpoint policy mirrors the same six ARNs.
- Execution-role versus task-role separation is consistent across all four stacks, and every execution role's `GetSecretValue` list matches 1:1 the secrets its own task definition injects.
- No task or Lambda reads the RDS master secret except db-admin; the gateway is injected only the app-user secret. The master secret is genuinely break-glass. There is no `iam:PassRole` to any workload role anywhere in the templates.

- 04 imports the **read-only** spend key; the write key is never exported (02:2115-2124). The portal task role's write surface is a single prefix (`identity/principal-emails/*`), verified against `identity.py`; it cannot touch `releases/`.
- `ActivitySubscriptionRole` and `BedrockPromptLoggingRole` are confused-deputy-hardened with `aws:SourceAccount` (and `SourceArn` plus the exact stream ARN for the latter).
- The two condition-scoped KMS grants (`AmpRemoteWriteKms`, the Grafana task-role query grant) are exactly right — `kms:ViaService=aps.<region>.amazonaws.com`, gated on the CMK being in use.

Network

- No `0.0.0.0/0` **ingress** exists anywhere in the four templates; all eight occurrences of that CIDR are egress. Every SG-to-SG arrow is a matched ingress/egress pair, and cross-stack SG writers are complete and symmetric with no orphaned half-rules.
- RDS is reachable by security-group reference only (01:196-214).
- There is no plaintext hop on any data path: no HTTP:80 listener, a single HTTPS:443 listener, `HealthCheckProtocol: HTTPS` explicit on all three target groups and CI-gated (`rules.guard:53-61`), `rds.force_ssl` with `sslmode=verify-full`, and the only cleartext is loopback within one Fargate network namespace.
- The ADOT sidecar is verifiably hop-free: no `PortMappings`, loopback receivers, and no collector security group anywhere.
- Endpoint policies are present and tightly scoped wherever the service supports them; the `ecs` omission is deliberate and disclosed (A7).
- ALB posture: internal and ipv4 (guard-enforced), `drop_invalid_header_fields`, FIPS TLS policy by default, access logs on, and `AssignPublicIp: DISABLED` on all services.

Application

- RS256 verification is correct and strict (`crypto.py:85-160`): the algorithm is pinned (both `none` and confusion attacks refused), JWKS keys filtered on `kty`/`use`, signature length and `s < n` checked, a full PKCS#1 v1.5 EM reconstruction compared in constant time, and `iss`/`aud` (scalar and list)/`exp`/`nonce` all enforced, with real-signature tests behind it.
- The OIDC code flow is complete: per-login `state` compared with `compare_digest`, S256 PKCE, a bound nonce, confidential-client Basic exchange, groups unioned from the ID token and userinfo, and non-member denials audited. Authorization is evaluated only after full token verification. JWKS refetch is throttled to 300 s on unknown `kid`, so forged random-kid tokens cannot amplify outbound requests.
- A synchronizer CSRF token guards every mutating POST, explicitly not relying on SameSite alone, with the reasoning written down.
- Hardening headers are applied to every response including redirects, errors, and static files: a CSP with no `unsafe-inline` or `unsafe-eval`, `nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`. Frame relaxation is confined to the guide/PDF route. No inline script or style exists; the dropdown mapping is an inert JSON island escaped with `|tojson`.
- Redirect following is disabled on all gateway calls (`gateway.py:35-45`), so `urllib` cannot replay an `Authorization` header to a `Location` host.
- Every request-controlled value is allowlisted before use: cost-center and team validated as a **pair**, platform validated server-side regardless of the JS path, shell and batch metacharacters rejected at boot and per request, S3 keys built only from configuration and validated values, identity-map keys charset-restricted so `../` cannot appear. Downloads stream without `Content-Length` so truncation is detectable, and the audit record is written **before** the stream starts.

- Money handling is exact integer-cents string arithmetic with strict format validation.
- The rotation Lambda's one string-built SQL statement (`ALTER ROLE` cannot take a bind parameter) inlines only alphanumeric generated passwords (`ExcludePunctuation=True`) and double-quotes its one dynamic identifier — a narrow, reasoned exception rather than a pattern.

Supply chain

- Release mirroring is properly fail-closed: the manifest signature is required unless `ALLOW_UNVERIFIED_MANIFEST=1` (two paths, both exit 3), per-platform sha256 is checked, and a mismatched binary is deleted rather than left staged (`mirror-claude-release.sh:44-121`).
- The Grafana plugin pin is single-sourced (`grafana-plugin.pin`) and verified on both sides of the air gap, with runtime signature verification and both grafana.com boot fetchers disabled (`03-observability.yaml:892-905`).
- The offline-build rule is genuinely implemented: zero network fetches outside `scripts/mirror/`; everything flows through `require_mirrored_file` / `verify_sha256`, which abort naming the exact mirror script to run on the egress host (`common.sh:66-88`).
- `mirror-base-images.sh` is the model implementation: platform pin plus architecture assertion, content-derived idempotent tags against IMMUTABLE repositories, ECR digest read-back with the manifest-list caveat handled, and a fail-closed CMK check with a named override.
- `ensure_ecr_repo` enforces IMMUTABLE tags including back-fill on existing repositories, scan-on-push, and CMK at creation.
- Image builds need no package manager or registry access at build time: committed `vendor/` wheels installed `--no-index`, and no yum/apt/npm/curl in any Dockerfile.
- Both installers are TOCTOU-safe and non-destructive: stage-then-verify the local copy, refuse root/SYSTEM, and preserve an unparseable `settings.json` rather than clobbering it — with real bats and Pester coverage of the merge functions.

Monitoring and audit

- `TreatMissingData` is correct and non-obvious: `breaching` on both cessation alarms (03:326, 375), with `OKActions` set.
- The intermittent-audit false-fire problem is handled by a default-off `ActivityLogsAlarmMinutes: 0` with in-template rationale — a reasoned default, not a bad one.
- The fail-closed telemetry chain is real, not aspirational: the sidecar is `Essential` with a `HealthCheck` against the loopback health-check extension, the gateway `DependsOn` it reaching HEALTHY, and `StopTimeout` is 120 s. The template itself states that container health is not proof of ingestion.
- Every audit destination is CMK-encrypted, pre-created in CloudFormation, and `Retain` — including the pgaudit group that RDS would otherwise auto-create with default encryption (01:251-258) and both portal groups (04:304-322). Every log group also carries a load-bearing `retention-policy: retain-on-teardown` tag.
- pgaudit is both useful and privacy-aware: `shared_preload` pgaudit with `ddl,role,write`, connection and disconnection logging, and `pgaudit.log_parameter = '0'` so bind values carrying user content stay out of the log (01:228-241).
- The portal audit record is written **before** the download stream starts (`downloads.py:129`), so a mid-stream abort cannot erase it; denials and admin actions are audited with `gateway_actor oidc:<sub>`.
- `om-runbooks.md` §9 and `monitoring-and-retention.md` are accurate rather than aspirational — every retention claim checks out against the templates.
- The Bedrock prompt-log destinations are deliberately unconditional with the reasoning recorded in-template (03:536-557), and `deploy-observability.sh:44-71` preflights the CMK key policy so Bedrock's misleading S3 error surfaces as its real KMS cause.

8. Verification approach and limitations

Verified against code. Every finding in §3–§5 cites a file and, for templates and application code, a line range that was read during the pass. IAM policies, key policies, trust policies, endpoint policies, bucket policies, security-group rules, alarm definitions, retention settings, the ADOT sidecar configuration, both Dockerfile sets, the mirror scripts, the deploy scripts, the portal Flask package, the db-admin Lambda, and both client installers were all read directly. A sample of the cited line ranges was independently re-confirmed while writing this document.

Verified by prior live exercise. Several posture statements rest on the pilot's live run rather than on code inspection alone: the loopback telemetry path (including that the gateway refuses a non-loopback plaintext forward URL and that its SSRF guard blocks loopback without the named override), the CMK-encrypted AMP query and remote-write paths, the metric-temporality behavior, and the device-flow admin path (exercised offline against a mirrored gateway build plus a throwaway Postgres and a fake RS256 issuer).

Doc- or web-verified only — treat as hypothesis. The following were not exercised in this pass and are flagged wherever they are load-bearing:

- The gateway binary's X-Forwarded-For parsing order (SA-2026-07-30). Everything else about the binary is observed through its configuration schema, not its source.
- Whether the gateway's refresh grant re-validates Okta groups (A9).
- Whether a CMK-encrypted Firehose DirectPut stream requires added KMS permissions for the CloudWatch Logs subscription principal (SA-2026-07-19) — this differs between Kinesis and Firehose and needs a live check before the change is made.
- The Bedrock prompt-log delivery data prefix (A4, SA-2026-07-58), which by construction can only be learned from a live delivery.
- The Linux client flow end to end, and the managed-settings path `/etc/claude-code/managed-settings.json`, which is verified against Anthropic's settings documentation only.
- Stack 04 has, as of this assessment, **no live deploy verification** of the streamed download at real size or the fingerprint page against the real ALB certificate.

Explicitly out of scope. No penetration testing, no fuzzing, no dependency CVE scan of the vendored wheels or base images (the ECR scan-on-push result is not reproduced here), and no review of the Claude apps gateway binary itself. Controls inherited from Okta, ZPA, and the AWS Landing Zone were not assessed; SA-2026-07-13 records that one such inheritance (CloudTrail) is currently undocumented.

Assessment-package caveat. SA-2026-07-01 is the standing example of why the code, not this document set, is authoritative: the package asserted a key-lifecycle property the code does not implement. Documentation-completeness gaps of that kind are enumerated separately in `ato-package-gaps.md`.

9. Related documents

Document	Relationship
`poam.md`	Remediation tracking for every finding in §3–§5, one row per ID, all Open. Accepted risks (§2) are deliberately **not** POA&M items.
`ato-package-gaps.md`	Documentation-completeness gaps in the ATO package itself — distinct from the system findings here, and not duplicated in either direction.
`architecture.md`	Diagrams, secrets/SG/encryption inventories. §10 of that document carries the short-form accepted-risk list; §2 here is the full form.
`network-access-controls.md`	The reachability inventory that findings SA-2026-07-05, -25, -26, and -30 refer to.
`conops.md`	Users, roles, operational and degraded modes; the fail-closed behaviors in A3 and A16 are described operationally there.
`../operations/monitoring-and-retention.md`	The authoritative alarm/retention inventory the monitoring findings are measured against.
`../operations/cost-controls.md`	§5 is the fail-closed spend-store recovery runbook referenced by A3 and SA-2026-07-14.

---

10. Post-assessment changes (appended; this assessment is point-in-time)

- **2026-08-07 — `WebFetch` removed from the managed deny list** (commit "SB -- Allowing WebFetch"). A8's description of the *applied* denies changes accordingly: the policy now denies `WebSearch` and `mcp__*` only, and `WebFetch` joins `Bash` (curl/wget) and `Agent` as a deliberately-allowed client-side web path, bounded by the same client-side Zscaler policy. The A8 acceptance rationale (residual web access bounded by Zscaler, not by tool denies) already covers this posture; `control-implementation.md` AC-20 and `client-config.md` §6c reflect it. A future assessment supersedes this note.